

Lab05-AddNewSyscall

目标

在linux内核中添加一个新的系统调用，并在用户模式下测试系统调用

工具

安装gcc编译器

代码块

```
1 sudo apt-get install build-essential
```

实验流程

1. Step0

第三节课完成自定义配置的内核：linux-6.17.1

2. Step1 (Linux kernel 6.2+)

在文件 include/linux/syscalls.h (第1220行)

代码块

```
1 #endif /* CONFIG_ARCH_HAS_SYSCALL_WRAPPER */之前，添加一行：  
2 asmlinkage long sys_schello(void);
```

3. Step2 (Linux kernel 6.2+)

kernel/sys.c 在文件SYSCALL_DEFINE0(gettid)函数之后（第1008行），添加如下行：

代码块

```
1 // 定义系统调用：无参数，返回 0  
2 SYSCALL_DEFINE0(schello)  
3 {  
4     printk(KERN_INFO "Hello new system call schello! Your ID\n");  
5     printk(KERN_INFO "Hello new system call schello! Hello 123456\n");  
6     return 0;  
7  
8 }
```

4. Step3 (Linux kernel 6.2+)

针对x86的64位系统，在arch/x86/entry/syscalls/syscall_64.tbl的最后一行之后添加：

代码块

```
1  470    common  schello          sys_schello
```

其中，470为最后一行开头的数字加1。

针对arm的64位系统，文件位置为：arch/arm64/tools/syscall_64.tbl，同样在最后一行添加上述代码。

5. Step4: 增加调用号

include/uapi/asm-generic/unistd.h中__NR_syscalls附近（895行之后），做如下修改，1）将470修改为471，2）增加两行有关schello的调用定义：

代码块

```
1  #define __NR_schello 470
2  __SYSCALL(__NR_schello, sys_schello)
3
4  #undef __NR_syscalls
5  #define __NR_syscalls 471
```

6. Step5: 编译

代码块

```
1  #make clean
2  make -j$(nproc) 2>&1 | tee log
3  sudo make modules_install
4  sudo make install
```

7. Step 6

重新启动：reboot

确认新内核是否成功运行

代码块

```
1  uname -a
2  // 注：显示和编译版本一致才为正确
```

8. Step 7

编写用户态测试程序testschello.c

代码块

```
1  #include <unistd.h>
2  #include <sys/syscall.h>
3  #include <sys/types.h>
4  #include <stdio.h>
5  int main(int argc, char *argv[])
6  {
7      syscall(470);
8      printf("ok! run the cmd in terminal: sudo dmesg | grep schello\n");
9      return 0;
10 }
```

9. Step 8

编译用户态测试程序testschello.c，并执行

代码块

```
1  # 编译
2  gcc -o testsc testschello.c
3  # 运行
4  ./testsc
5  sudo dmesg | grep schello
```

10. 实现上述实例，并输出linux kernel版本号和你的“学号”，并撰写实验报告